

Dynamic Cloud-Edge-End Collaboration Reconfiguration for Federated Learning under Resource and Privacy Constraints

Tingting Yang^a, Wenbo Zhang^a, Shaohua Liu^a, Lihua Zhang^a, Xinyuan Lin^a, Qian Liu^a

^aBeijing University of Posts and Telecommunications, Beijing, China

Abstract

Hierarchical federated learning reduces direct cloud communication, but fixed workflows cannot adapt computation, aggregation, and privacy protection to heterogeneous devices and changing system conditions. We propose a unified cloud-edge-end collaboration reconfiguration framework that dynamically determines execution placement and aggregation topology under heterogeneous resource conditions, while every formal global release follows a fixed client-level DP and HE protection contract. Link blocks and link flows provide a unified collaboration space in which heterogeneous federated learning workflows become comparable decisions. A joint model evaluates latency, communication, privacy consumption, encrypted aggregation overhead, and estimated convergence error cost. The resulting multiobjective problem is solved through bounded Pareto search and an ideal point Tchebycheff criterion. Experiments on CIFAR 10 with ResNet 18 and Fashion MNIST with LeNet 5 examine privacy feasibility, test accuracy, communication, and system time including the measured decision cost.

Keywords: Internet of Things, hierarchical federated learning, cloud-edge-end collaboration, collaboration reconfiguration, differential privacy, device heterogeneity, resource constraints

1. Introduction

Federated learning (FL) enables multiple end devices to collaboratively train a model by keeping data local and uploading model parameters and gradient based updates [1]. With the development of artificial intelligence for IoT (AIoT), mobile edge computing, and intelligent sensing systems, training data are increasingly generated by cameras, mobile terminals, vehicles, and industrial sensors. Cloud-edge-end hierarchical federated learning (HFL) has therefore been introduced to reduce long distance communication between end devices and the cloud through edge aggregation [2]. It also enables multiple edge domains to participate in a shared cloud level training process.

However, HFL in cloud-edge-end environments still faces two major challenges. First, end device data are usually non-IID, and significant distribution differences among devices can lead to model drift and performance degradation under simple weighted aggregation [3, 4]. Second, IoT devices are system heterogeneous. Low power cameras and sensors may be unable to support full training of a large deep model, whereas edge servers and cloud servers usually have stronger computation and memory resources [5]. If all devices are forced to follow the same training process, weak and delayed devices may become stragglers and may even be excluded by resource constraints.

Existing work has addressed these issues from several directions, but most studies improve a fixed HFL procedure and leave dynamic assignment of training tasks across the end, edge, and cloud layers insufficiently explored. Personalized HFL mitigates non-IID drift through edge side personalization, similarity aware aggregation, and model decoupling [6, 7]. Resource constrained and wireless FL optimize local up-

date frequency, communication rounds, energy consumption, and wireless resource allocation [5, 8]. Cloud-edge-end FL studies further consider heterogeneous model collaboration, multiple server edge aggregation, and split training mechanisms [9, 10, 11]. Nevertheless, most existing methods are built around a specific collaborative procedure with a fixed hierarchical structure. They pay insufficient attention to the complete collaboration decision, namely which model blocks should be executed at the end, edge, and cloud layers, which aggregation flow should be used, and how this task assignment should interact with privacy mechanisms.

Privacy protection further complicates this decision. DP limits the influence of one client by clipping and perturbing model updates, but it may degrade convergence. Adaptive DP methods tune privacy budgets, clipping thresholds, and client selection to improve the privacy utility tradeoff [12, 13, 14]. HE, secure aggregation, and privacy preserving offloading have also been studied in FL and cloud-edge-end systems [15, 16, 17]. Different collaborative modes expose model updates at different trust boundaries. For example, split training keeps intermediate features and backward gradients inside the associated end and edge execution domain, whereas cloud aggregation requires protected model updates to cross that domain boundary. Therefore, the communication, computation, and estimated convergence error costs of privacy mechanisms should be modeled together with collaborative mode selection.

The central difficulty is that privacy protection, training utility, and system efficiency are tightly coupled. Stronger privacy protection can increase computation and communication overhead and can introduce clipping distortion and perturbation

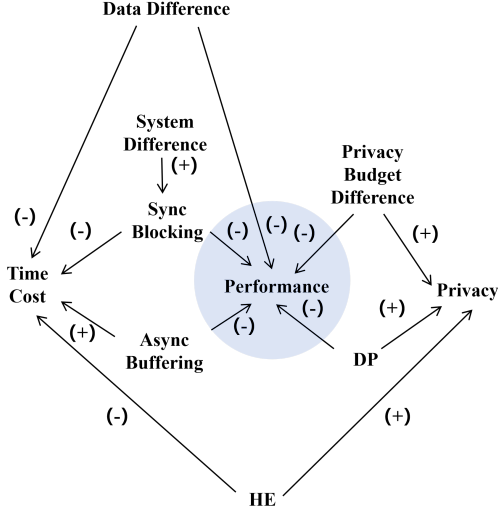


Figure 1: Coupled tradeoff among privacy protection, training utility, and efficiency.

noise. Moving computation among the end, edge, and cloud layers can relieve resource limited devices, but it also changes cross layer transmission, synchronization latency, and the objects exposed to privacy mechanisms. Consequently, improving one aspect may degrade another, and a global cloud-edge-end decision must balance privacy protection, training efficiency, and convergence behavior. Figure 1 illustrates this coupled design objective.

Motivated by this challenge, this paper studies dynamic reconfiguration of cloud-edge-end federated learning workflows under resource and privacy constraints. Each configuration specifies model execution, computation placement, and aggregation flow across the end, edge, and cloud layers, while the privacy release contract is fixed independently of the selected collaboration mode. The main contributions are summarized as follows.

- We propose a dynamic cloud-edge-end collaboration reconfiguration framework that adapts execution placement and mode-specific logical communication while preserving a fixed trusted-edge global-release topology and one mandatory client-level DP release contract.
- We introduce a unified collaboration space representation based on link blocks and link flows. It describes model execution, transmitted objects, aggregation locations, and return paths, while a fixed HE confidentiality layer protects cloud-facing aggregate traffic.
- We establish a joint model for latency, communication, encrypted aggregation overhead, and estimated convergence error cost under the common DP release mechanism. HE and DP are modeled with distinct roles rather than as interchangeable privacy choices.
- We formulate collaboration reconfiguration as a constrained multiobjective problem. A bounded Pareto search constructs feasible global decisions, and an

ideal point Tchebycheff criterion selects the executed configuration.

The framework is evaluated through 100 round CIFAR 10 training with ResNet 18 under heterogeneous devices and edge level label skew.

2. Related Work

2.1. HFL Under Heterogeneous Environments

HFL introduces edge servers between end devices and the cloud to reduce direct cloud communication pressure through edge pre aggregation [2]. Resource aware FL studies reduce stragglers and communication overhead by adjusting client selection, local update frequency, pruning, and model compression under limited computation, energy, and bandwidth resources [18, 19, 5, 8]. Personalized HFL further mitigates performance degradation under non-IID data through edge side personalized models, similarity aggregation, and model decoupling [6, 7]. These methods show that HFL can improve training from resource scheduling, communication control, and data heterogeneity adaptation. However, when end device resources differ across computation, memory, and communication capabilities, using the same training flow for all clients can still cause straggler blocking in conventional synchronous aggregation and prevent weak devices from participating effectively. This paper jointly considers end device computation constraints, memory constraints, communication overhead, and model split positions during collaboration reconfiguration. Dynamic selection itself may also introduce reconfiguration overhead. Recent multi job FL studies explicitly model switching cost in client selection and reduce it through block wise selection, where the selected client subset is kept unchanged across multiple rounds [20, 21]. This observation motivates the use of a strategy update period in our framework. Collaborative modes are re evaluated only at update points, which limits unnecessary mode reconfiguration while preserving adaptability to resource and link changes.

2.2. Cloud-Edge-End Collaborative Federated Learning

HFL is a representative form of cloud-edge-end collaborative FL, in which end devices, edge nodes, and the cloud jointly execute computation, aggregation, and communication tasks. As model size and system heterogeneity increase, fixed full end device training followed by edge aggregation becomes less suitable for resource limited IoT devices. FedAgg [9] uses bridge samples and online distillation to enable models of different scales at the end, edge, and cloud layers to collaborate. SFETEC [11] introduces split federated learning into Thing Edge Cloud environments and partitions model computation across layers to reduce end device computation load. Cloud-edge-end task offloading studies also combine FL, privacy protection, and task efficiency [15]. These studies show the value of collaboration among the three layers, but they usually assume a fixed collaboration structure. In contrast, this paper studies how each device should select a collaborative

mode under given resource and privacy conditions. The need to balance reconfiguration gain and reconfiguration cost is also consistent with edge computing task migration studies. In mobile edge computing, migration can reduce service delay when users move away from the original edge server, but it also consumes backhaul transmission and reconfiguration resources [22]. Load aware task migration further models migration cost as a combination of communication, queuing, and computation costs, and notes that migration cost can be related to physical distance between edge nodes [23]. These studies suggest that dynamic cloud-edge-end collaboration should account for both the benefit of changing execution paths and the overhead caused by such changes.

2.3. Privacy Preserving HFL

Model parameters, gradients, and intermediate representations in federated training may leak information about private data. Differential privacy (DP), homomorphic encryption (HE), and secure aggregation are therefore widely used in FL. For HFL, group local DP provides personalized privacy through sampling, randomized response, and shuffling [12]. Adaptive DP FL improves convergence and model utility through dynamic privacy budget allocation, adaptive clipping, and client selection [13]. Other privacy preserving HFL studies combine client level DP, secure aggregation, and CKKS style HE for data space scenarios [16]. Related fog FL studies combine hierarchical aggregation, personalization, and DP for IoT intrusion detection [17]. These studies mainly focus on the security guarantee, budget allocation, and accuracy degradation of a particular privacy mechanism. The joint modeling of privacy induced communication cost, computation latency, estimated convergence error cost, and cloud-edge-end collaboration reconfiguration remains insufficiently studied.

The added capability is the joint reconfiguration of a complete federated learning workflow. Existing categories optimize selected resources, training steps, or privacy parameters within a prescribed collaboration structure. The proposed collaboration space instead makes complete workflows with different execution locations, aggregation paths, transmitted objects, and privacy mechanisms comparable under one feasible decision model.

3. Model Construction

3.1. Cloud-Edge-End Collaborative Link Modeling

We consider a three tier cloud-edge-end collaborative FL system composed of end devices, edge nodes, and a cloud node. Let $\mathcal{N}$ denote the set of end devices. Each end device $i \in \mathcal{N}$ selects one collaborative mode h_i from a candidate mode set $\mathcal{H}$. In this paper, a collaborative mode specifies the model split position, the layer that executes each model block, the aggregation flow, and the return path of aggregation results across the end, edge, and cloud layers. Given a global model M , different modes induce the partition $M = M_L^{h_i} \cup M_E^{h_i} \cup M_C^{h_i}$, where $M_L^{h_i}$, $M_E^{h_i}$, and $M_C^{h_i}$ are the submodels executed at the end, edge, and cloud layers, respectively. Weaker devices use split positions

closer to the input layer and offload more computation toward upper layers, whereas stronger devices can undertake a larger proportion of local training.

Each end device and its associated edge node form a trusted execution domain. Intermediate representations, logits, backward gradients, and model updates exchanged inside this domain are not exposed to the cloud adversary. The cloud is honest but curious, and update transmissions from an end device or edge node to the cloud cross the trust boundary. Cloud-facing aggregate updates are protected by HE during transport and cloud aggregation. Independently of the selected executable mode, every formal global model release is produced by a client-level DP mechanism before the cloud obtains any decryptable aggregate. HE therefore provides confidentiality and does not replace DP. Raw samples remain at the end device throughout training.

Definition 1 (Collaborative Link Block). Given a collaborative mode h_i , a collaborative link block is the local training step and cross layer interaction associated with end device i before an aggregation event. Formally, $B_i^{h_i} = (d_i, \ell_i, g_i, \mathcal{E}_{B,i}^{h_i})$, where d_i is the participating end device, $\ell_i \in \{L, E, C\}$ indicates the update submission layer from which the local model update is uploaded to the aggregation node before aggregation, $g_i \in \{A_e, A_c\}$ is the aggregation node that receives the update, and $\mathcal{E}_{B,i}^{h_i}$ is the set of intermediate feature, backward gradient, and model update transmission links caused by split training. According to ℓ_i , $B_i^{h_i}$ is classified as B_L , B_E , or B_C , corresponding to a local update submitted for aggregation from the end, edge, or cloud layer, respectively.

Definition 2 (Collaborative Link Flow). A collaborative link flow is a complete training link formed by at least one collaborative link block, the aggregation node, and the model return process. A one level aggregation collaborative link flow can be abstracted as

$$\text{Link flow } d_i \xrightarrow{B_i^{h_i} \times L_B} \ell_i \xrightarrow{\text{upload}} a \xrightarrow{\text{download}} q_i. \quad (1)$$

Here, $\xrightarrow{B_i^{h_i} \times L_B}$ denotes that end device i performs L_B link blocks under collaborative mode h_i , with the local update submitted for aggregation from layer ℓ_i . a is the selected aggregation node, and q_i denotes the specific return receiver for device i . In the modes studied here, q_i is the end device d_i when the returned state is used locally, and it is the associated edge node when the returned submodel is kept at the edge side. If two level aggregation exists, the collaborative link flow also contains an edge aggregation node A_e , the cloud aggregation node A_c , and the final return to the end device.

$$\text{Link flow } [d_i \xrightarrow{B_i^{h_i} \times L_B} \ell_i \xrightarrow{\text{upload}} A_e] \times E_B \xrightarrow{\text{upload}} A_c \xrightarrow{\text{download}} A_e \xrightarrow{\text{download}} d_i. \quad (2)$$

E_B denotes the number of edge level aggregation loops completed at A_e before the edge aggregated update is uploaded to A_c .

Table 1: Representative Capability Comparison

Method category	Execution ment	place- ogy	Aggregation topol- ogy	Transmitted ob- jects	Privacy mecha- nisms	Joint reconfigura- tion
Conventional HFL [2]	Fixed		Fixed hierarchy	Model update	Not a decision	No
Resource aware FL [5, 8]	Fixed		Fixed	Model update	Not a decision	No
Split cloud-edge-end FL [11]	Fixed split		Fixed	Feature, gradient, update	Not a decision	No
Privacy preserving HFL [12, 16]	Fixed		Fixed	Mechanism specific	DP or HE in a fixed workflow	No
Proposed framework	Adaptive		Adaptive	Feature, gradient, update	Joint DP and HE decision	Yes

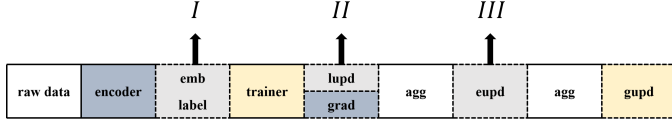


Figure 2: Abstract training flow and candidate interaction interfaces in cloud-edge-end collaborative FL. Interfaces I, II, and III represent feature transmission, split training update and gradient interaction, and aggregation update transmission, respectively. The labels emb, lupd, grad, eupd, and gupd denote intermediate embedding, local update, backward gradient, edge aggregated update, and global aggregated update.

Definition 3 (Cloud-Edge-End Collaborative Link Graph). For a collaborative mode h_i , its training process is represented by the directed graph $\mathcal{G}^{h_i} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ contains end, edge, and cloud nodes and $\mathcal{E}$ contains their interaction links. Each link is denoted by $\lambda = (u, v, b)$, where u and v are the source and target nodes and b is the interaction type. The link size, rate, and latency are treated as link attributes in the cost model.

Based on the abstract training flow illustrated in Fig. 2, the unified representation describes seven basic modes over the end (L), edge (E), and cloud (C) layers. The interfaces mark candidate interaction boundaries from shallow to deep stages. A deeper boundary keeps more computation on the end side, whereas an earlier boundary offloads more computation toward upper layers.

$$\mathcal{H}_{\text{base}} = \{\text{LIE}, \text{LIC}, \text{LIIE}, \text{LIIC}, \text{LIEIIC}, \text{LIEIIC}, \text{LIEIIC}\}. \quad (3)$$

LIC sends intermediate representations directly to the cloud and is therefore excluded under the stated trust model. The executable mode set is $\mathcal{H} = \mathcal{H}_{\text{base}} \setminus \{\text{LIC}\}$. The symbols L , E , and C indicate the update submission layer of a collaborative link block, while I, II, and III denote the collaborative interaction interfaces defined in Fig. 2.

Definition 4 (Collaborative Link Transmission Indicator Matrix). Let the node order be (L, E, C) . The transmission types are $\mathcal{B} = \{\text{emb}, \text{grad}, \text{upd}\}$. Here, emb denotes intermediate embedding transmission, grad denotes backward gradient transmission in split training, and upd denotes local, edge aggregated, or global aggregated model update transmission. For mode h_i and transmission type $b \in \mathcal{B}$, the collaborative link transmission indicator matrix is

$$\mathbf{I}_b^{h_i}(u, v) = \begin{cases} 1, & \text{mode } h_i \text{ transmits type } b \text{ from } u \text{ to } v, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

For example, for one complete LIEIIC link flow,

$$\mathbf{I}_{\text{emb}}^{\text{LIEIIC}} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad \mathbf{I}_{\text{grad}}^{\text{LIEIIC}} = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad (5)$$

$$\mathbf{I}_{\text{upd}}^{\text{LIEIIC}} = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

The nonzero entries indicate transmission directions only. The corresponding protection-role matrix is

$$\mathcal{O}_b^{h_i}(u, v) = \begin{cases} \{\text{trusted}\}, & \mathbf{I}_b^{h_i}(u, v) = 1, (u, v) \in \{(L, E), (E, L)\}, \\ \{\text{HE}\}, & \mathbf{I}_b^{h_i}(u, v) = 1, b = \text{upd}, v = C, \\ \{\text{none}\}, & \mathbf{I}_b^{h_i}(u, v) = 1, \text{model return}, \\ \emptyset, & \mathbf{I}_b^{h_i}(u, v) = 0. \end{cases} \quad (6)$$

Thus, $\mathbf{I}_b^{h_i}$ says whether a transmission exists, whereas $\mathcal{O}_b^{h_i}$ gives its fixed trust or transport role. DP is not a link-level alternative in this matrix because it is applied once at the global release boundary for every executable mode. For LIEIIC, the update protection-role matrix is

$$\mathcal{O}_{\text{upd}}^{\text{LIEIIC}} = \begin{pmatrix} \emptyset & \emptyset & \emptyset \\ \{\text{none}\} & \emptyset & \{\text{HE}\} \\ \emptyset & \{\text{none}\} & \emptyset \end{pmatrix}. \quad (7)$$

The entries with $\{\text{none}\}$ are model return links.

The selected mode determines the transmitted objects and their directions. Embedding and gradient exchanges remain inside the trusted end-edge domain, while the cloud-facing aggregate path uses HE. We retain π_i in the latency notation as the unique protection map induced by mode h_i ; hence $\Pi(h_i) = \{\pi_i^{\text{fusion}}\}$ is a singleton and is not a selector decision variable. Model return links are counted for communication latency only and use none.

Collaborative link flows of the seven modes. The following expressions summarize the seven basic mode templates. They describe the taxonomy; the formal Method 2 release overlay and its independent-stage semantics are specified immediately below. Detailed transmitted objects and the fixed trust/transport roles are determined by $\mathbf{I}_b^{h_i}$ and $\mathcal{O}_b^{h_i}$.

- **LIE** $\mathcal{F}(\text{LIE}) = B_E \times L_B \rightarrow A_e \rightarrow \text{end}$.
- **LIIE** $\mathcal{F}(\text{LIIE}) = B_L \times L_B \rightarrow A_e \rightarrow \text{end}$.

- **LIC** $\mathcal{F}(\text{LIC}) = B_C \times L_B \rightarrow A_c \rightarrow \text{end}$.
- **LIIC** $\mathcal{F}(\text{LIIC}) = B_L \times L_B \rightarrow A_c \rightarrow \text{end}$.
- **LIEIC** $\mathcal{F}(\text{LIEIC}) = B_E \times L_B \rightarrow A_c \rightarrow A_e \rightarrow \text{end}$.
- **LIEIIC** $\mathcal{F}(\text{LIEIIC}) = [B_E \times L_B \rightarrow A_e \rightarrow \text{end}] \times E_B \rightarrow A_c \rightarrow A_e \rightarrow \text{end}$.
- **LIIEIIC** $\mathcal{F}(\text{LIIEIIC}) = [B_L \times L_B \rightarrow A_e \rightarrow \text{end}] \times E_B \rightarrow A_c \rightarrow A_e \rightarrow \text{end}$.

In the fused protocol, $E_B = 3$ denotes three consecutive private training stages for each client in LIEIIC and LIIEIIC. Each stage contains three local epochs, so these modes execute nine local epochs per FL round; the other executable modes use one stage of three epochs. A client continues its own model and optimizer state across stages, starting from the previous released global model. There is no same-round cross-client aggregate feedback. Clipping and distributed Gaussian noise are applied once to the final client contribution, after all its stages. As a supplementary fairness control, we also provide an equal-optimizer-work setting that caps every collaboration mode at the same total local-epoch budget. This control is disabled for the main Method-2 experiments and is reported separately; it tests whether gains persist when the additional private-stage optimizer work is removed. Every executable mode, including LIE and LIIE, then joins the same release overlay: one trusted noisy packet per edge domain, ciphertext-only cloud aggregation, authorized custodian decryption, and global update return. The basic templates' edge-only termination and repeated edge-aggregate feedback are not executed in this overlay. The flow evaluator charges one terminal edge aggregation per edge and one global aggregation; E_B scales private training work, not release count. This independence condition is required by the client-replacement analysis below.

The corresponding cloud-edge-end collaborative link graphs of the seven modes are illustrated in Fig. 3. Each sub-figure corresponds to one basic collaboration-mode template. These graphs define the mode taxonomy and the mode-specific logical communication charged by the cost model; they do not replace the fixed formal release overlay described above. Orange links show link block execution repeated L_B times. Green self loops show the edge aggregation loop before global aggregation, repeated E_B times. Black self loops show the final global aggregation. Black arrows show update upload and model return paths among end devices, edge nodes, and the cloud. In the basic taxonomy, modes LIE and LIIE terminate at the edge side, modes LIC and LIIC use cloud aggregation directly, LIEIIC uploads edge-side updates to the cloud and returns through the edge side, and LIEIIC and LIIEIIC include the E_B edge loop before cloud aggregation. In the formal mainline experiments, these terminal aggregation routes are not executed literally: the selected mode determines computation stages and mode-specific logical communication, while every executable mode joins the same trusted-edge aggregation and single global DP/HE release topology.

3.2. Joint Cost and Utility Modeling

After the link topology is determined, each client selects only a collaborative mode h_i . The protection map $\pi_i = \pi_i^{\text{fusion}}(h_i)$ is induced by the selected mode and the fixed release contract rather than optimized independently. The selected mode must satisfy the computation and memory limits of the end device, while the joint mode profile determines the system level latency and estimated convergence error cost. The modeling below is organized in five parts, including end side resource feasibility, path level latency, buffered aggregation cost, global release privacy feasibility, and convergence error cost estimation.

3.2.1. End Side Resource Feasibility

Let n_i be the number of local samples processed by device i in one round, and let L_B be the number of local link block iterations. Under mode h_i , $\chi(h_i)$ denotes the profiled computation workload required by the end layer to process one sample. The subscript L denotes the local end device layer. The single round local computation workload is $W_{i,L}^{\text{comp}}(h_i) = L_B n_i \chi(h_i)$.

The value of $\chi(h_i)$ depends on the model blocks assigned to the end device and is obtained through offline profiling. When detailed operator level profiling is unavailable, it is approximated as $\chi(h_i) \approx \kappa_c m_i(h_i)$, where $m_i(h_i)$ is the parameter scale assigned to device i , and κ_c is a calibrated conversion coefficient. This provides a lightweight complexity estimate for candidate mode screening and is in line with resource aware FL latency models [24].

Let F_i denote the effective computing rate of device i . The end side computation latency is $T_{i,L}^{\text{comp}}(h_i) = W_{i,L}^{\text{comp}}(h_i)/F_i$. This follows the common wireless FL latency model, where local computation time depends on local iterations, local data size, per sample computation workload, and device computing rate. Given the maximum allowable computation time τ_i^{max} , the computation budget is $Q_{\text{comp},i} = F_i \tau_i^{\text{max}}$.

The memory requirement under mode h_i is $M_i^{\text{mem}}(h_i) = s_{\text{state}} m_i(h_i) + a_i(h_i)$. This model follows the common separation between parameter related model states and residual training memory [25]. Here, s_{state} is the profiled per parameter model state memory coefficient, covering parameters, gradients, optimizer states such as Adam momentum and variance, and other persistent training states such as mixed precision parameter copies. The term $a_i(h_i)$ denotes residual memory that is not captured by the parameter scale, including activations and temporary training buffers.

Let $Q_{\text{mem},i}$ denote the available memory of device i . The resource feasible mode set is

$$\mathcal{H}_i^{\text{res}} = \{h_i \in \mathcal{H} \mid W_{i,L}^{\text{comp}}(h_i) \leq Q_{\text{comp},i}, M_i^{\text{mem}}(h_i) \leq Q_{\text{mem},i}\}. \quad (8)$$

3.2.2. Path Level Latency

We first define the general link and path latency terms. A representative complete path is then illustrated in Fig. 4. Let $R^t = (R_{u,v}^t)_{u,v \in \mathcal{V}}$ be the cloud-edge-end link rate matrix in round t . The communication model is link specific. For an end edge pair (i, j) , the rate $R_{i,j}^t$ and the base latency $T_{i,j}^0$ may differ from

Table 2: Cloud-Edge-End Collaboration Modes

Mode	Execution structure	Basic-template aggregation	Protected upload	Available mechanism
LIE	End and edge split	Edge	Internal embedding and gradient	Trusted execution + global DP/HE releas
LIC	End and cloud split	Cloud	Embedding and gradient	Excluded in this threat model
LIIE	Local training	Edge	Internal client update	Trusted execution + global DP/HE releas
LIIC	Local training	Cloud	Client contribution	Global DP + HE transport
LIEIC	End and edge split	Cloud through edge	Client contribution	Global DP + HE transport
LIEIIC	Repeated trusted execution stages	Edge then cloud	Client contribution	Global DP + HE transport
LIIEIIC	Repeated trusted execution stages	Edge then cloud	Client contribution	Global DP + HE transport

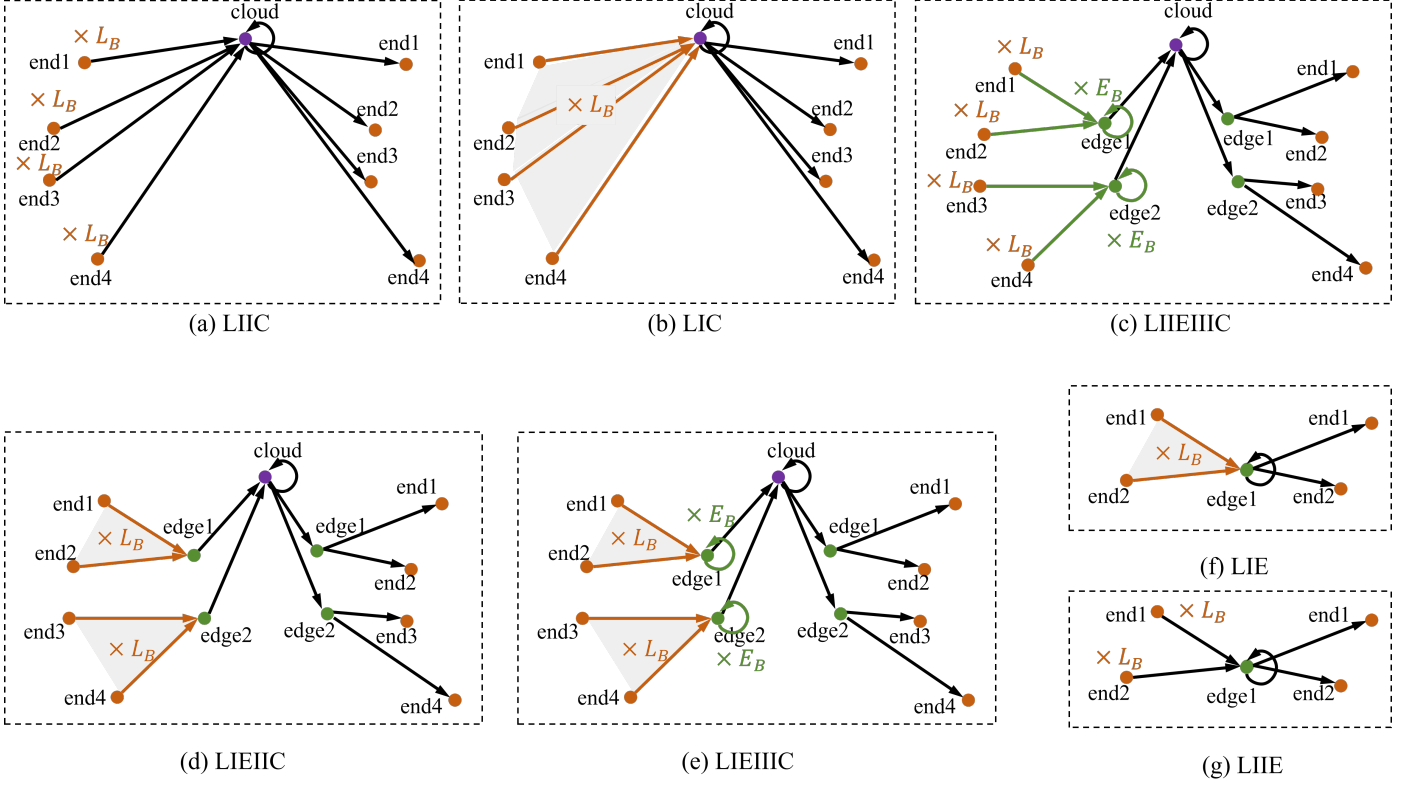


Figure 3: Cloud-edge-end collaborative link graphs of the seven collaborative modes.

those of other device edge pairs. These parameters capture heterogeneous physical distance, wireless channel quality, interference, and congestion. Therefore, two end devices connected to the same edge server can have different communication latencies, and the same device connected to different edge servers can also experience different communication latencies [22, 23]. For an interaction link $\lambda = (u, v, b)$, its raw transmission size is $S_{\lambda}^t(h_i)$. Let $\psi_{\lambda}(h_i, \pi_i)$ denote the actual trust/transport mechanism on link λ under mode h_i , where $\pi_i = \pi_i^{\text{fusion}}(h_i)$ is the unique induced protection map rather than an independent policy choice. It returns the trust/transport mechanism fixed by the induced protection map and uses none for links that only transmit returned models.

$$\psi_{\lambda}(h_i, \pi_i) = \begin{cases} \pi_i(\lambda), & \lambda \in \mathcal{E}_{\text{priv}}^{h_i}, \\ \text{none}, & \lambda \notin \mathcal{E}_{\text{priv}}^{h_i}. \end{cases} \quad (9)$$

Thus, a model return link uses none even when its object type is upd. The effective transmission size is $\tilde{S}_{\lambda}^t(h_i, \pi_i) = \alpha(\psi_{\lambda}(h_i, \pi_i))S_{\lambda}^t(h_i)$, where $\alpha(\cdot)$ is the communication inflation

factor induced by the fixed induced protection mechanism. For none, $\alpha(\text{none}) = 1$. For DP, $\alpha(\text{DP}) \approx 1$. For HE, ciphertext encoding and packing increase the transmitted payload, so $\alpha(\text{HE}) > 1$ [16]. In practice, $\alpha(\text{HE})$ can be obtained by offline profiling as the ratio between the serialized ciphertext payload size and the original plaintext tensor size. The communication latency is

$$T_{\lambda}^{\text{link},t}(h_i, \pi_i) = \frac{\tilde{S}_{\lambda}^t(h_i, \pi_i)}{R_{\lambda}^t} + D_{\lambda}^0 + T_{\lambda}^{\text{priv},t}(\psi_{\lambda}(h_i, \pi_i)), \quad (10)$$

where $R_{\lambda}^t = R_{u,v}^t$ for $\lambda = (u, v, b)$, D_{λ}^0 is the fixed base delay, and $T_{\lambda}^{\text{priv},t}$ is the link endpoint privacy processing latency. For none, $T_{\lambda}^{\text{priv},t}(\text{none}) = 0$. Round level network fluctuation is represented by the time varying link rate $R_{u,v}^t$.

Using this link level term, the path level link flow latency of end device i , denoted by $T_i^t(h_i, \pi_i)$, is decomposed into execution, communication, waiting, aggregation, and switching components.

The transmission indicator matrices build the communication part of the link flow as $\mathcal{E}_{\text{flow}}^{h_i} = \{\lambda = (u, v, b) \mid b \in \mathcal{B}, \mathbf{I}_b^h(u, v) = 1\}$. Thus, nonzero entries in $\mathbf{I}_b^h$ identify the communication links induced by mode h_i . The subset $\mathcal{E}_{\text{priv}}^{h_i}$ contains the links in $\mathcal{E}_{\text{flow}}^{h_i}$ that require privacy protection. Let $N_\lambda(h_i)$ be the number of times link λ is executed in one complete link flow. For a link executed inside a local link block, $N_\lambda(h_i) = L_B$. If the local link block is nested inside an edge aggregation loop, the count becomes $N_\lambda(h_i) = L_B E_B$. For links executed once per edge aggregation loop, $N_\lambda(h_i) = E_B$. For final aggregation upload and model return links outside repeated loops, $N_\lambda(h_i) = 1$. The path level latency is decomposed as

$$T_i^t(h_i, \pi_i) = T_i^{\text{exec},t}(h_i) + \sum_{\lambda \in \mathcal{E}_{\text{flow}}^{h_i}} N_\lambda(h_i) T_\lambda^{\text{link},t}(h_i, \pi_i) + T_i^{\text{wait},t}(h_i) + T_i^{\text{agg},t}(h_i, \pi_i) + T_i^{\text{sw},t}. \quad (11)$$

Here, $T_i^{\text{exec},t}$ denotes only the computation latency of the sub-models executed at the end, edge, and cloud layers. It is coupled with the selected link flow through h_i , but it does not include cross layer transmission delay. That delay is accounted for separately by the summation of $T_\lambda^{\text{link},t}$ over $\mathcal{E}_{\text{flow}}^{h_i}$. $T_i^{\text{wait},t}$ denotes the waiting latency before upper layer aggregation is triggered, and $T_i^{\text{agg},t}$ denotes the aggregation latency assigned to the link flow. Changing the collaborative mode may require execution path rescheduling and model state movement, similar to task migration and switching cost modeling in edge computing and wireless FL [22, 23, 20, 21, 26]. The term $T_i^{\text{sw},t}$ denotes this mode reconfiguration latency in round t , and we model it as

$$T_i^{\text{sw},t} = \eta_h \mathbb{I}(h_i^t \neq h_i^{t-1}) + \eta_p \frac{\|\mathbf{p}(h_i^t) - \mathbf{p}(h_i^{t-1})\|_1}{2}, \quad (12)$$

where $\eta_h \geq 0$ is the fixed time cost incurred when the selected collaborative mode changes, similar to the unit switching cost used in dynamic FL client selection [20]. $\eta_p \geq 0$ is the maximum variable time cost associated with model state placement change, motivated by edge service migration models where migration cost is proportional to the migrated service state size [26]. Both coefficients are configurable or profile based system parameters. To quantify the placement-change term in $T_i^{\text{sw},t}$, we define the normalized model state placement vector of mode h as

$$\mathbf{p}(h) = \frac{1}{Z_h} [|\mathcal{M}_L^h|, |\mathcal{M}_E^h|, |\mathcal{M}_C^h|], \quad (13)$$

where $Z_h = |\mathcal{M}_L^h| + |\mathcal{M}_E^h| + |\mathcal{M}_C^h|$. $\mathcal{M}_L^h$, $\mathcal{M}_E^h$, and $\mathcal{M}_C^h$ are the model state portions placed at the end, edge, and cloud layers under mode h , respectively, and $|\cdot|$ denotes profiled model state size. This normalization gives $\mathbf{1}^\top \mathbf{p}(h) = 1$, so $\|\mathbf{p}(h_i^t) - \mathbf{p}(h_i^{t-1})\|_1 / 2 \in [0, 1]$. The normalized distance estimates the fraction of model state whose placement changes between two consecutive selected modes.

3.2.3. Buffered Aggregation Cost

This subsection specifies the waiting and aggregation terms used in the path latency model under buffered aggregation. To

mitigate synchronization blocking caused by slow devices, aggregation is triggered after the required number of updates has arrived. For edge aggregator j , let N_j be the number of associated end devices and $\rho_{\text{buf}} \in (0, 1]$ be the buffered aggregation participation threshold. The number of clients needed to trigger aggregation is $K_j = \lceil \rho_{\text{buf}} N_j \rceil$. Let T_i^{lupd} be the time needed for device i to complete the collaborative update process before upload. Let $T_{i,j}^{\text{comm},t}(h_i, \pi_i)$ denote the communication latency of the update upload from device i to edge aggregator j , obtained from $T_\lambda^{\text{link},t}$ for the corresponding link $\lambda = (i, j, \text{upd})$. The arrival time at edge j is $x_{i,j}^t = T_i^{\text{lupd}}(h_i, \pi_i) + T_{i,j}^{\text{comm},t}(h_i, \pi_i)$. Sorting arrival times as $x_{(1),j}^t \leq \dots \leq x_{(N_j),j}^t$, let $S_j^t(\mathbf{x}) = \{i_{(1)}, \dots, i_{(K_j)}\}$ be the first K_j arrived clients selected into the buffered aggregation. The edge aggregation starts at $X_j^t(\mathbf{x}) = x_{(K_j),j}^t = \max_{i \in S_j^t(\mathbf{x})} x_{i,j}^t$, and client i waits $W_{i,j}^{\text{agg},t} = \max(0, X_j^t(\mathbf{x}) - x_{i,j}^t)$. If multiple updates arrive exactly at the trigger time $X_j^t(\mathbf{x}) = x_{(K_j),j}^t$, all tied updates are admitted, so $|S_j^t(\mathbf{x})|$ may exceed K_j . The edge aggregation time is $T_j^{\text{agg},t} = \beta_j \sum_{i \in S_j^t(\mathbf{x})} \widetilde{S}_{i,j}^{\text{upd},t} + T_j^{\text{fix}}$, where β_j is the aggregation processing coefficient of edge aggregator j . For the update upload link $\lambda = (i, j, \text{upd})$, $\widetilde{S}_{i,j}^{\text{upd},t} = \widetilde{S}_\lambda^t(h_i, \pi_i)$, i.e., the effective payload size of client i 's update received by edge aggregator j after the fixed protection map is applied. The coefficient β_j can be obtained by offline profiling with varying update counts and payload sizes, and T_j^{fix} denotes the fixed aggregation overhead. Protection processing, including fixed HE overhead on cloud-facing aggregate paths, is accounted for in the link endpoint term $T_\lambda^{\text{priv},t}$.

The same construction is applied to every aggregation group induced by a complete candidate profile $\mathbf{x} = \mathbf{h}_N$. The induced protection map is fixed once each h_i is chosen. Clients are grouped by their selected aggregation endpoint and compatible link flow. Each edge buffer admits its earliest required arrivals. An admitted edge aggregate that reaches the cloud then becomes a cloud arrival, and the cloud buffer applies the same rule to its incoming client or edge objects. Let $A_i^t(\mathbf{x})$ denote the source clients admitted to the buffer associated with client i 's selected path under profile $\mathbf{x}$, and let $A^t(\mathbf{x}) = \bigcup_i A_i^t(\mathbf{x})$ be the complete admitted client set. The system latency $T_{\text{sys}}^t(\mathbf{x})$ is the latest completion time of the triggered terminal aggregation and model return events. The link rates, resource state, topology, and previous profile are fixed during one evaluation, while $\mathbf{x}$ determines the path choices and arrival times. Therefore, every candidate profile produces its own $A_i^t(\mathbf{x})$ and $T_{\text{sys}}^t(\mathbf{x})$. When $\rho_{\text{buf}} = 1$, every active buffer waits for all its members and the same procedure reduces to full admission.

Fig. 4 illustrates how the general latency decomposition in (11) is instantiated in a representative complete end-edge-cloud link flow. The directional communication terms in the figure, such as $T_{i,j}^{\text{comm},t}$, $T_{j \rightarrow c}^{\text{comm},t}$, $T_{c \rightarrow j}^{\text{comm},t}$, and $T_{j \rightarrow i}^{\text{comm},t}$, are shorthand instances of $T_\lambda^{\text{link},t}$ for the corresponding directed links and transmitted objects. Marker ① denotes the local model training time of the end device, corresponding to $T_i^{\text{lupd}}(h_i, \pi_i)$. Marker ② denotes the communication time for uploading the transmission object of the selected link flow to the associated edge server, corresponding to $T_{i,j}^{\text{comm},t}(h_i, \pi_i)$. The transmission object is de-

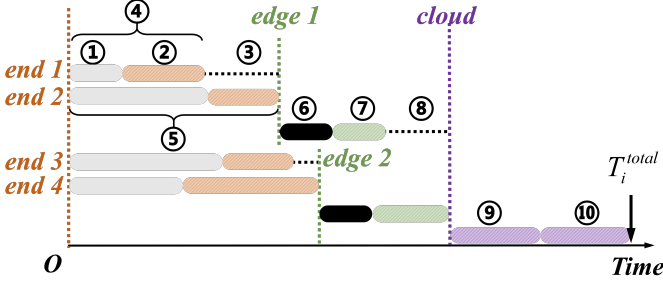


Figure 4: Path level link flow latency decomposition for an end device in cloud-edge-end collaborative FL.

terminated by $b \in \mathcal{B}$. Marker ③ denotes the waiting time after the upload is completed and before the edge server starts aggregation, corresponding to $W_{i,j}^{\text{agg},t}$. Marker ④ denotes the arrival time $x_{i,j}^t$ at the edge server. Marker ⑤ denotes the start time $X_j^t(\mathbf{x})$ of local edge aggregation, which is determined by the latest arrival among the selected buffered aggregation set $S_j^t(\mathbf{x})$. Marker ⑥ denotes the edge aggregation time, corresponding to $T_j^{\text{agg},t}$. Marker ⑦ denotes the communication time for uploading the edge aggregated model to the cloud server, corresponding to $T_{j \rightarrow c}^{\text{comm},t}$. Marker ⑧ denotes the waiting time after the edge server completes uploading and before the cloud server starts global aggregation, corresponding to $W_{i,c}^{\text{agg},t}$. Marker ⑨ denotes the cloud global aggregation and model return stage, including the cloud aggregation time $T_c^{\text{agg},t}$ and the cloud to edge download time $T_{c \rightarrow j}^{\text{comm},t}$. Marker ⑩ denotes the communication time for returning the updated global model from the edge server to the end device, corresponding to $T_{j \rightarrow i}^{\text{comm},t}$.

For this representative path, $T_i^t(h_i, \pi_i)$ can be expanded as

$$T_i^t(h_i, \pi_i) = T_i^{\text{lupd}}(h_i, \pi_i) + T_{i,j}^{\text{comm},t}(h_i, \pi_i) + W_{i,j}^{\text{agg},t} + T_j^{\text{agg},t} + T_{j \rightarrow c}^{\text{comm},t} + W_{j,c}^{\text{agg},t} + T_c^{\text{agg},t} + T_{c \rightarrow j}^{\text{comm},t} + T_{j \rightarrow i}^{\text{comm},t}. \quad (14)$$

The expanded form is shown only for the representative path in Fig. 4. For any selected mode h_i , $T_i^t(h_i, \pi_i)$ accumulates the terms on the complete link flow induced by that mode, including the model return required to prepare the device for the next round.

3.2.4. Privacy Budget Accounting

The formal privacy unit is one client contribution. In round t , every participating client starts from the previously released global state and computes an update u_i^t without same-round private feedback from other clients. The contribution is clipped as

$$\bar{u}_i^t = u_i^t \min\left(1, \frac{C_u}{\|u_i^t\|_2}\right). \quad (15)$$

Let q_i^t be the public normalized aggregation weight actually used for the formal release, with $\sum_i q_i^t = 1$. Under client replacement adjacency, the sensitivity of the weighted aggregate is

$$S_t = 2C_u \max_i q_i^t. \quad (16)$$

The trusted edge domains generate distributed Gaussian noise shares whose weighted sum has distribution $\mathcal{N}(0, (\sigma_u S_t)^2 I_d)$.

The shares are added before HE encryption. The honest-but-curious cloud therefore receives ciphertexts representing already-noisy contributions, performs ciphertext aggregation only, and a designated trusted edge custodian decrypts the already-noisy aggregate. HE protects confidentiality on the cloud path; it does not replace the DP mechanism applied to the released model.

We use RDP accounting for repeated Gaussian releases [27]. Because the noise standard deviation is calibrated as $\sigma_u S_t$, one formal global release adds

$$\Delta R_{i,\text{upd}}^t(\alpha) = \frac{\alpha}{2\sigma_u^2} \quad (17)$$

for every client whose contribution is covered by that release. The corresponding $(\epsilon, \delta_{\text{upd}})$ value after the tentative release is

$$\epsilon_{i,\text{upd}}^{t,+} = \min_{\alpha > 1} \left[R_{i,\text{upd}}^t(\alpha) + \frac{\alpha}{2\sigma_u^2} + \frac{\log(1/\delta_{\text{upd}})}{\alpha - 1} \right]. \quad (18)$$

A release is permitted only when this value does not exceed the fixed target $\bar{\epsilon}_{i,\text{upd}}$. In the formal experiments the participation roster is fixed and all clients participate in each release; data-dependent admission or weighting would require a separate privacy analysis.

The total privacy target does not change with the number of training rounds. With one formal aggregate release per communication round, $K_{\text{upd}}^{\text{max}}(T) = T$, and the noise multiplier is selected before training to satisfy

$$\min_{\alpha > 1} \left[\frac{T\alpha}{2\sigma_u^2} + \frac{\log(1/\delta_{\text{upd}})}{\alpha - 1} \right] \leq \bar{\epsilon}_{i,\text{upd}}. \quad (19)$$

Increasing the privacy horizon therefore recalibrates σ_u while keeping the reported total privacy target fixed. The implementation reports this release accountant separately from any legacy link-level diagnostic ledger. The current guarantee is for the stated aggregate-release mechanism under the fixed public participation assumptions; a full end-to-end DP claim for arbitrary adaptive protocol decisions is not established.

3.2.5. Estimation of Convergence Error Cost

We next estimate the convergence error cost under the common privacy release mechanism. For every executable mode, the client contribution $u_i^t = \text{vec}(\theta_i^t - w^t)$ is first clipped according to (15). Clipping introduces the gradient-unit distortion

$$\mathbf{b}_{\text{clip},i}^t = \frac{\bar{u}_i^t - u_i^t}{\eta_{\text{lr}} L_B}, \quad (20)$$

$$\mathcal{B}_{\text{clip},i}^t = \mathbb{E}[\|\mathbf{b}_{\text{clip},i}^t\|_2^2 | w^t].$$

This term is zero when clipping is inactive and increases with the amount removed from client updates. Execution placement alone does not create an additional accuracy penalty: two modes that produce the same client contribution distribution share the same clipping and privacy-noise terms.

For the formal release, let $q_i^t(\mathbf{h})$ denote the normalized public aggregation weight of client i under the selected collaboration profile $\mathbf{h}$. In the main experiments all clients participate,

and the implementation uses the same normalized weights in sensitivity calibration and model aggregation. Define

$$q_{\max}^t(\mathbf{h}) = \max_i q_i^t(\mathbf{h}), \quad S_t = 2C_u q_{\max}^t(\mathbf{h}). \quad (21)$$

The released aggregate is

$$\bar{\mathbf{u}}^t = \sum_i q_i^t(\mathbf{h}) \bar{\mathbf{u}}_i^t + \xi^t, \quad \xi^t \sim \mathcal{N}(0, \sigma_u^2 S_t^2 I_d). \quad (22)$$

Consequently the aggregate DP-noise contribution is

$$\mathbb{E} \|\xi^t\|_2^2 = d\sigma_u^2 S_t^2. \quad (23)$$

The distributed edge noise shares are an implementation of this single aggregate noise term; they are not separate privacy releases. Likewise, HE changes communication and cryptographic computation cost but does not remove the clipping or DP-noise term.

We assume that the original global objective $F(w)$ is G -smooth and satisfies the μ -PL condition $\|\nabla F(w)\|_2^2 \geq 2\mu(F(w) - F(w^*))$. Combining local stochastic variance, clipping distortion, and the common aggregate DP noise motivates the selection-time error proxy. A convenient separable client term is

$$\begin{aligned} B_i^t &= \frac{3}{2\mu} \mathcal{B}_{\text{clip},i}^t, \\ V_i^t &= \frac{G\eta_{\text{r}} L_B}{\mu} \sigma_{\text{local}}^2. \end{aligned} \quad (24)$$

The profile-level proxy adds the common release variance $d\sigma_u^2 S_t^2$ and the cloud-fusion/system terms defined below. Thus privacy cost is not inferred from a mode label or a DP-versus-HE choice; it changes only when a selected training variable changes the actual contributions, aggregation weights, clipping statistics, or release sensitivity.

3.3. Global Multi-Objective Collaborative Optimization

The previous subsection defines the resource feasibility, path level latency, privacy budget, and convergence error cost terms for one device. Cloud-edge-end FL ultimately trains a system level global model. If each device independently selects the locally fastest and lowest estimated cost mode, cross edge information fusion may be insufficient, especially under non-IID data. Let $\mathbf{h}_N = [h_1, \dots, h_N]$ denote the global collaboration-mode vector. The protection map is fixed by the fused release contract and is therefore omitted from the decision vector.

Path level latency T_i^t describes the time contribution generated by the path selected for device i at decision epoch t . A complete profile changes path choices, arrival order, aggregation timing, and model-return events. The general evaluator retains the buffer notation $A_i^t(\mathbf{x})$, but the formal fused configuration sets the aggregation fraction to one, so the release roster is the fixed full client set rather than a data-dependent admitted subset. For $\mathbf{x} = \mathbf{h}_N$, the evaluator therefore produces the system latency $T_{\text{sys}}^t(\mathbf{x})$ while every client remains in the formal release.

For the formal fused mainline, the release roster is fixed and the aggregation fraction is one. Therefore every executable

mode contributes one client update to the same formal global release, even when its internal collaboration flow performs trusted edge aggregation. Hence $A_C^t(\mathbf{x}) = \mathcal{N}$ and the diagnostic cloud fusion ratio is $r_C^t(\mathbf{x}) = 1$. We retain $\Psi_C^t = \xi/(1 + \varepsilon)$ in the implementation for compatibility with the general evaluator, but it is constant and does not change the ordering of fused profiles.

The common aggregate-noise contribution expressed in the same optimization scale is

$$V_{\text{DP}}^t(\mathbf{x}) = \frac{G\eta_{\text{r}} L_B}{\mu} \frac{d\sigma_u^2 S_t(\mathbf{x})^2}{\eta_{\text{r}}^2 L_B^2}. \quad (25)$$

The system-level estimate of convergence error cost is

$$\hat{\Omega}_{\text{sys}}^t(\mathbf{x}) = \sum_{i \in N} [q_i^t B_i^t + (q_i^t)^2 V_i^t] + V_{\text{DP}}^t(\mathbf{x}) + \Psi_C^t. \quad (26)$$

The future training loss is unavailable during selection, so this quantity is a selection-time proxy. The privacy term is common to modes that produce the same public weights and clipping statistics; execution placement alone does not create an artificial accuracy difference.

Let $\mathbf{x} = \mathbf{h}_N$ denote a complete system decision. Its global feasible set is

$$\mathcal{X}^t = \{\mathbf{x} \mid h_i \in \mathcal{H}_i^{\text{res}}, \epsilon_{i,\text{upd}}^{t,+} \leq \bar{\epsilon}_{i,\text{upd}}, \forall i \in N\}. \quad (27)$$

The global multi objective problem minimizes system latency and the system level estimate of convergence error cost.

$$\underset{\mathbf{x} \in \mathcal{X}^t}{\text{minimize}} \quad \mathbf{f}^t(\mathbf{x}) = \begin{bmatrix} T_{\text{sys}}^t(\mathbf{x}) \\ \hat{\Omega}_{\text{sys}}^t(\mathbf{x}) \end{bmatrix}. \quad (28)$$

The Pareto optimal decision set and its objective space Pareto front are

$$\begin{aligned} \mathcal{X}_{\text{p}}^t &= \{\mathbf{x} \in \mathcal{X}^t \mid \nexists \mathbf{x}' \in \mathcal{X}^t, \mathbf{x}' < \mathbf{x}\}, \\ \mathcal{F}_{\text{p}}^t &= \{\mathbf{f}^t(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}_{\text{p}}^t\}. \end{aligned} \quad (29)$$

Here $<$ denotes Pareto dominance. The latency range is $[T_{\min}^t, T_{\max}^t]$, and the estimated cost range is $[\hat{\Omega}_{\min}^t, \hat{\Omega}_{\max}^t]$, both evaluated over $\mathcal{X}_{\text{p}}^t$. The normalized objectives are

$$\begin{aligned} \bar{T}^t(\mathbf{x}) &= \frac{T_{\text{sys}}^t(\mathbf{x}) - T_{\min}^t}{T_{\max}^t - T_{\min}^t + \varepsilon_n}, \\ \bar{\Omega}^t(\mathbf{x}) &= \frac{\hat{\Omega}_{\text{sys}}^t(\mathbf{x}) - \hat{\Omega}_{\min}^t}{\hat{\Omega}_{\max}^t - \hat{\Omega}_{\min}^t + \varepsilon_n}. \end{aligned} \quad (30)$$

Here $\varepsilon_n > 0$ prevents division by zero. The reference point $\mathbf{z}^{r,t} = [z_T^{r,t}, z_{\Omega}^{r,t}]^T$ is set to the computed ideal point $[T_{\min}^t, \hat{\Omega}_{\min}^t]^T$. Its normalized form is $\bar{\mathbf{z}}^{r,t} = [\bar{z}_T^{r,t}, \bar{z}_{\Omega}^{r,t}]^T$. Using the normalized Tchebycheff distance to the reference or ideal point [29], we define

$$\begin{aligned} D_{\infty}^t(\mathbf{x}) &= \max\{|\bar{T}^t(\mathbf{x}) - \bar{z}_T^{r,t}|, |\bar{\Omega}^t(\mathbf{x}) - \bar{z}_{\Omega}^{r,t}|\}, \\ \mathbf{x}^* &\in \arg \min_{\mathbf{x} \in \mathcal{X}_{\text{p}}^t} D_{\infty}^t(\mathbf{x}). \end{aligned} \quad (31)$$

Thus, the selected profile minimizes its largest normalized deviation from the reference or ideal point. If several profiles attain the same minimum distance, the solver returns one minimizer. Real test accuracy is used in experiments to evaluate the effect of the estimated convergence error cost objective.

4. Algorithm

Each device first constructs S_i by retaining the executable modes h_i that satisfy the resource constraints. The common release accountant is checked once for the prospective round; it is not a per-mode mechanism choice. A complete global profile selects one mode from every S_i and is evaluated by $(T_{\text{sys}}^t, \widehat{\Omega}_{\text{sys}}^t)$.

For a small instance, the exact solver evaluates all $M = \prod_{i \in \mathcal{N}} |S_i|$ global profiles. Here, $|S_i|$ is the number of feasible candidates for device i , and the product over all devices gives the total number of joint profiles. Sorting the resulting pairs by increasing latency and scanning the estimated cost gives the exact Pareto set. Applying (31) then gives one ideal point Tchebycheff solution over the exact Pareto set. If one global evaluation costs C_{eval} , this procedure costs $O(MC_{\text{eval}} + M \log M)$. Its exponential value of M limits it to small scale validation.

For full scale selection, a bounded search initializes a profile archive and expands each retained profile by replacing one device decision at a time. A deterministic exploration queue containing at most K_B evaluated intermediate profiles allows the search to pass through a temporarily dominated profile. The method maintains a bounded archive containing at most K_P profiles that are non dominated among the evaluated candidates. Here, $K_P \geq 2$ is the prescribed maximum archive size and controls the tradeoff between Pareto front coverage and search complexity. Because only a subset of $\mathcal{X}^t$ is explored, the resulting archive $\widehat{\mathcal{X}}_P^t$ approximates the exact Pareto set $\mathcal{X}_P^t$. The exact and approximate solvers use the same objective evaluation, normalization, and ideal point Tchebycheff selection. Algorithm 1 presents the bounded solver.

To describe the search cost, let $K_{\text{cand}} = \max_i |S_i|$ be the largest number of candidates available to one device. From one retained profile, changing one device at a time generates at most $N(K_{\text{cand}} - 1)$ neighboring profiles. Since the archive and exploration queue contain at most $K_P + K_B$ profiles, one expansion evaluates at most $L \leq (K_P + K_B)N(K_{\text{cand}} - 1)$ neighboring profiles. Evaluating them costs $O(LC_{\text{eval}})$, and sorting them for Pareto filtering costs $O(L \log L)$. Therefore, at most L_{max} expansions cost $O(L_{\text{max}}L(C_{\text{eval}} + \log L))$. Direct evaluation of the two system objectives aggregates the terms of N devices, so $C_{\text{eval}} = O(N)$ in the worst case. The implementation caches each candidate flow and updates the weighted error terms and affected buffer groups incrementally for a one device replacement.

The search is performed only at the strategy update points determined by S_P . At other rounds, the previous profile is reused. The latency caused by changing collaborative modes is included in T_i^t .

The fused mainline does not apply a DP-versus-HE substitution safeguard. Result DP is a mandatory release constraint and HE is the fixed confidentiality layer for the cloud aggregation path, so neither can be selected as a replacement for the other. The fixed release accountant is configured for the prospective round before selection, while its authoritative cohort and budget check is performed immediately before the global release. The common release-noise term is evaluated from the actual public aggregation weights via (16); because

this protection contract is common to all feasible modes, it does not create an artificial mode-specific utility preference. Legacy packet-protection experiments may retain mechanism-specific diagnostics, but those controls are outside the formal fused candidate set.

Because every executable mode sets the global-release requirement, the final protected release contains the complete public roster:

$$A_C^t(\mathbf{x}) = \mathcal{N}, \quad r_C^t(\mathbf{x}) = 1. \quad (32)$$

The general implementation retains the edge-coverage checker as a fail-closed assertion, but (32) makes it automatically satisfied in the formal fused configuration.

With full participation, the edge-normalized aggregation rule reduces to the public sample-proportional weight

$$q_i^t = \frac{n_i}{\sum_{k \in \mathcal{N}} n_k}, \quad \Delta w^t = \sum_{i \in \mathcal{N}} q_i^t \Delta w_i^t. \quad (33)$$

The same public weights are used for clipping-sensitivity calibration in (16). Each noisy weighted contribution is encrypted before the untrusted cloud sums ciphertexts, and only the trusted custodian decrypts the already-noisy aggregate.

Algorithm 1 Bounded Pareto Solver for Collaboration Reconfiguration

Require: Device set $\mathcal{N}$, executable resource-feasible modes $\mathcal{H}_i^{\text{res}} \subseteq \mathcal{H}$, fixed global release RDP state $R_{\text{upd}}^t(\alpha)$, target $(\bar{\epsilon}_{\text{upd}}, \delta_{\text{upd}})$, round system state, path latency and buffer models, $K_P, K_B, I_{\text{max}}, S_P$, and $\mathbf{x}^{t-1}$

Ensure: Selected global collaboration profile $\mathbf{x}^* = \mathbf{h}_{\mathcal{N}}^*$

1: **Preparation**

2: Verify that the fixed release accountant is configured for the prospective round; the authoritative cohort/budget check is performed immediately before the global release.

3: For every $i \in \mathcal{N}$, construct $S_i = \{h_i \mid h_i \in \mathcal{H}_i^{\text{res}}\}$.

4: $\mathcal{X}^t = \left\{ \mathbf{x} \in \prod_{i \in \mathcal{N}} S_i \mid \mathbf{x} \text{ satisfies (32)} \right\}, \quad \mathbf{x} = \mathbf{h}_{\mathcal{N}}.$

5: **if** t is not a strategy update point and $\mathbf{x}^{t-1} \in \mathcal{X}^t$ **then**

6: **return** $\mathbf{x}^{t-1}$.

7: **end if**

8: **Objective Evaluation**

9: For every generated profile $\mathbf{x}$, sort its path arrivals at each edge and cloud buffer and obtain $A_i^t(\mathbf{x})$ and $T_{\text{sys}}^t(\mathbf{x})$.

10: Compute $q_i^t(\mathbf{x})$, $r_C^t(\mathbf{x})$, and $\widehat{\Omega}_{\text{sys}}^t(\mathbf{x})$ from the admitted cloud updates using (21) and (26).

11: **Approximate Pareto Search**

12: Construct $\mathbf{x}_T$, $\mathbf{x}_{\Omega}$, and three deterministic intermediate profiles from normalized local objective weights.

13: Move any infeasible initial profile into $\mathcal{X}^t$ by deterministic one device replacements.

14: Retain the non dominated initial profiles as $\mathcal{A}_0$ and set $\mathcal{B}_0 = \emptyset$.

15: **for** $r = 1, \dots, I_{\text{max}}$ **do**

16: Generate all unvisited feasible profiles $\mathcal{V}_r \subseteq \mathcal{X}^t$ that differ from a profile in $\mathcal{A}_{r-1} \cup \mathcal{B}_{r-1}$ at one device.

17: Evaluate $(T_{\text{sys}}^t, \widehat{\Omega}_{\text{sys}}^t)$ for every $\mathbf{x} \in \mathcal{V}_r$.

18: $\mathcal{A}_r = \{\mathbf{x} \in \mathcal{A}_{r-1} \cup \mathcal{V}_r \mid \nexists \mathbf{x}' \in \mathcal{A}_{r-1} \cup \mathcal{V}_r, \mathbf{x}' < \mathbf{x}\}$.

19: **if** $|\mathcal{A}_r| > K_P$ **then**

```

20:   Retain both objective endpoints and  $K_p - 2$  approximately
      uniform profiles in latency order.
21: end if
22:   Retain at most  $K_B$  newly evaluated profiles outside  $\mathcal{A}_r$  with the
      smallest normalized ideal point distance as  $\mathcal{B}_r$ .
23:   if  $\mathcal{A}_r = \mathcal{A}_{r-1}$  and  $\mathcal{B}_r = \emptyset$  then
24:     break
25:   end if
26: end for
27: Set  $\widehat{\mathcal{X}}_p^t \leftarrow \mathcal{A}_r$ .
28: Ideal Point Tchebycheff Selection
29: Compute  $T_{\min}^t, T_{\max}^t, \widehat{\Omega}_{\min}^t, \widehat{\Omega}_{\max}^t$  over  $\widehat{\mathcal{X}}_p^t$ .
30:  $\bar{T}^t(\mathbf{x}) = \frac{T_{\text{sys}}^t(\mathbf{x}) - T_{\min}^t}{T_{\max}^t - T_{\min}^t + \varepsilon_n}$ ,  $\bar{\Omega}^t(\mathbf{x}) = \frac{\widehat{\Omega}_{\text{sys}}^t(\mathbf{x}) - \widehat{\Omega}_{\min}^t}{\widehat{\Omega}_{\max}^t - \widehat{\Omega}_{\min}^t + \varepsilon_n}$ .
31: Set  $\mathbf{z}^{r,t}$  to  $[T_{\min}^t, \widehat{\Omega}_{\min}^t]^\top$ , and apply the same normalization to obtain
       $(\bar{z}_T^{r,t}, \bar{z}_\Omega^{r,t})$ .
32:  $D_\infty^t(\mathbf{x}) = \max\{|\bar{T}^t(\mathbf{x}) - \bar{z}_T^{r,t}|, |\bar{\Omega}^t(\mathbf{x}) - \bar{z}_\Omega^{r,t}|\}$ .
33:  $\mathbf{x}^* \in \arg \min_{\mathbf{x} \in \widehat{\mathcal{X}}_p^t} D_\infty^t(\mathbf{x})$ .
34: return  $\mathbf{x}^*$ .

```

The main symbols used in the collaboration reconfiguration model and solver are summarized in Table 3.

5. Experiments

5.1. Experimental Setup

We evaluate the proposed method with real model training supported by the system simulator. The cloud-edge-end system contains 100 end devices, 10 edge nodes, and one cloud node. The main comparison trains only the classification head of a pretrained ResNet 18 model on CIFAR 10 for 100 communication rounds under moderate edge label skew. Each round uses three local epochs per private stage and a learning rate of 0.01. LIEIIC and LIIEIIC use three stages; the other executable modes use one. Three runs use random seeds 42, 43, and 44. Fashion MNIST with LeNet 5 provides a second dataset and model setting. The main parameters are listed in Table 4.

The encrypted aggregator performs ciphertext addition without ciphertext multiplication. The coefficient modulus therefore contains one 50 bit data prime and one 40 bit special prime. Its total of 90 bits is below the 218 bit limit reported by SEAL for polynomial degree 8192 at the 128 bit security level.

The fixed link, switching, and aggregation profiles used by the analytical latency model are listed in Table 5.

The configured value $S_{\text{emb}} = 1.6$ MB is the payload assigned to one embedding transmission event in the analytical link model. The corresponding backward gradient event uses the same payload profile. This value is a reproducible simulation input rather than a per sample or per layer tensor size. The ResNet 18 update payload in the formal configuration contains the 5130 trainable classifier parameters stored as 32 bit floating point values, giving 20520 bytes before encryption. The CKKS processing profile uses 4 MB as its reference granularity and scales linearly with the actual update payload.

The pretrained model uses ResNet18_Weights.DEFAULT from torchvision. The formal CIFAR 10 configuration is

resnet18_pretrained_head: only the 512-to-10 classifier weight and bias (5130 parameters) are trained. The feature extractor and BatchNorm parameters and running statistics remain frozen. Client-level clipping and Gaussian noise operate on the complete classifier update; frozen backbone parameters are neither perturbed nor included in the encrypted update. Thus the unencrypted float32 update contains 20520 bytes. This scope is distinct from finetuning layer3 and layer4. The split point is placed after layer2, as shown in Table 6. The same parameter partition is used by model execution, transmitted update construction, and aggregation.

For each seed, a seeded permutation selects 12000 CIFAR 10 training samples and 2000 test samples. Images use channel means (0.4914, 0.4822, 0.4465) and standard deviations (0.2023, 0.1994, 0.2010). Under the main moderate edge label skew, each label is shared equally across two adjacent edge domains with cyclic wraparound, and each edge therefore contains two neighboring labels. The samples assigned to each edge are divided among its ten associated clients. The extreme edge label skew variant used for stress testing instead assigns each label to one edge domain. Each client then reserves 20% of its assigned samples for validation.

Client computation and memory factors are independently drawn from $\mathcal{U}(0.5, 2.0)$, and edge computation factors are drawn from $\mathcal{U}(2/3, 1.5)$. Lower computation factors indicate faster execution. For direction $u \rightarrow v$, the simulated rate in round t is

$$R_{u,v}^t = \bar{R}_{u,v}[1 + 0.2 \sin(t/3)]\zeta_{u,v}^t, \quad \zeta_{u,v}^t \sim \mathcal{U}(0.75, 1.25). \quad (34)$$

The base rates are 5 MB/s for end to edge links, 2.2 MB/s for end to cloud links, and 8 MB/s for edge to cloud links. Reverse links use the same base rate. These values define a reproducible synthetic network scenario rather than a physical network measurement.

The update noise multiplier is calibrated from the total privacy target for 100 global releases, one per FL round, giving $\sigma_u = 6.9112003$ for $(\epsilon, \delta) = (8, 10^{-5})$. Three local epochs occur before each release; local epochs do not themselves consume additional release events. Each client starts from the previous released global model, clips its classifier contribution at $C_u = 0.1$, and receives a distributed Gaussian noise share before encryption. Trusted edges combine these already-noisy contributions using the same public weights as the release accountant. Every formal round uses real SEAL, encrypts the complete trainable update, and releases only the authorized noisy aggregate. The ciphertext-only cloud worker is a separate process; the trusted custodian keeps the secret key. This is local process separation, not a physical deployment. All methods use the same data partition, optimizer, epochs per stage, and edge normalized aggregation rule in (33). The main comparison uses three random seeds, 42, 43, and 44. Controlled studies use the same seeds, 42, 43, and 44, and change only the parameter or component under evaluation. The key ablation uses the three main-comparison seeds 42, 43, and 44.

Every run stores the complete configuration, runtime environment, exact client partition, generated system profiles,

Table 3: Main Notation

Symbol	Meaning
$\mathcal{N}, N$	End device set and number of end devices
$\mathcal{H}_{\text{base}}, \mathcal{H}, \mathcal{H}_i^{\text{res}}$	Abstract mode set, executable mode set, and resource feasible mode set of client i
$\Pi(h_i), \pi_i$	Singleton fixed protection-map set induced by mode h_i , and its unique element; not an optimization variable in the fused mainline
$\psi_\lambda(h_i, \pi_i)$	Fixed trust/transport mechanism induced on link λ ; DP is applied at the common global release boundary rather than selected per link
$s_i = h_i, S_i$	One collaboration-mode decision of client i , and the feasible mode candidate set
L_B, E_B	Number of local link block loops and number of edge aggregation loops before cloud aggregation
$Q_{\text{comp},i}, Q_{\text{mem},i}$	Local computation budget and memory upper bound of client i
$R_{i,\text{upd}}^t(\alpha)$	Client RDP state for update DP at order α
$(\bar{\epsilon}_{i,\text{upd}}, \delta_{\text{upd}})$	Fixed total privacy target for the update channel
$W_{i,L}^{\text{comp}}, M_i^{\text{mem}}$	Computation workload and memory requirement of client i under a selected mode
$\chi(h_i), F_i$	Mode dependent sample computation workload and fixed computing rate of client i
$\epsilon_{i,\text{upd}}^{t,+}$	Projected update epsilon after one formal global aggregate release
T_i, T_{sys}	Path level link flow latency of client i , and system latency of one triggered aggregation event
$C_i^{\text{sw},t}, S_P$	Mode switching overhead in round t , and strategy update period for limiting reconfiguration frequency
K_P, K_B, I_{max}	Pareto archive limit, deterministic exploration queue limit, and maximum search expansions
$S_j^t(\mathbf{x}), S_j^t(\mathbf{x}) $	Set of client updates selected into edge aggregator j 's buffered aggregation under profile $\mathbf{x}$, and the number of selected updates
$A_i^t(\mathbf{x}), A^t(\mathbf{x})$	Admitted clients in the buffer associated with client i 's path, and complete admitted client set under profile $\mathbf{x}$
S_{agg}^t	Privacy processed update payload size used in aggregation computation
B_i^t, V_i^t, q_i^t	Client bias cost, variance cost, and effective cloud aggregation weight
V_{DP}^t, S_t	Common aggregate DP-noise error term and client-replacement sensitivity of the formal release
$\bar{\Omega}_{\text{sys}}^t$	System level estimate of convergence error cost
Ψ_C^t, r_C^t	Cloud fusion insufficiency penalty and cloud fusion ratio
$N_e, N_{e,A}^t$	Full edge data mass and admitted cloud data mass
$\rho_{\text{buf}}, \rho_{\text{PL}}$	Buffered aggregation participation threshold and PL based convergence contraction factor
$\eta_{\text{lr}}, \sigma_{\text{local}}^2$	Learning rate and local stochastic gradient variance
$C_u, \Delta_u, \mathcal{B}_{\text{clip},i}^t$	Update clipping threshold, client replacement sensitivity, and clipping bias estimate

Table 4: Default Simulation Parameters

Parameter	Value	Parameter	Value
End and edge nodes	100 end devices and 10 edge nodes	Training rounds	$T = 100$
Training and test samples	12000 and 2000	Random seeds	42, 43, and 44
Client computation and memory factors	Independent $\mathcal{U}(0.5, 2.0)$	Edge computation factor	$\mathcal{U}(2/3, 1.5)$
Logical link block reference	$L_B = 5$	Independent stages in multilevel modes	$E_B = 3$
Fixed total DP target	$\bar{\epsilon}_{\text{upd}} = 8.0$	DP slack parameter	$\delta_{\text{upd}} = 10^{-5}$
Base transmission size	$S_{\text{emb}} = 1.6 \text{ MB}, S_{\text{upd}} = 0.02052 \text{ MB}$	HE expansion factor	$\alpha_{\text{HE}} = 8.04$
CKKS polynomial degree and scale	8192 and 2^{40}	CKKS coefficient modulus bits	[50, 40]
CKKS reference profile for a 4 MB update	32.14 MB and 0.90 s	CKKS key preparation	Measured per global release
Base link rates	end to edge 5, end to cloud 2.2, edge to cloud 8 MB/s	Rate variation	sinusoidal amplitude 0.2 and uniform jitter 0.25
Aggregation fraction	$\rho_{\text{buf}} = 1.0$	Strategy update period	$S_P = 1$
Update clip norm (CIFAR 10 head)	$C_u = 0.1$	Auto calibrated noise multiplier	$\sigma_{\text{upd}} = 6.911$
Pareto archive, beam, and expansions	$K_P = 16, K_B = 4, I_{\text{max}} = 50$	Cloud fusion penalty	$\xi = 0.2, \epsilon = 0.05$
Training hardware	Intel i7 11700F, GTX 1660 Ti 6 GB, 31.8 GB RAM	Software	Python 3.10.9, PyTorch 2.5.1, torchvision 0.20.1, SEAL 4.0.0

Table 5: Additional Profiling Parameters

Parameter	Value used in the experiments
Base link delay Δ_λ^0	End to edge 0.015 s, end to cloud 0.040 s, and edge to cloud 0.010 s. Reverse links use the corresponding value.
Switching coefficients	$\eta_h = 0.02 \text{ s}$ and $\eta_p = 0.08 \text{ s}$
Edge aggregation profile	$\beta_j = 0.01 \text{ s/MB}$ and $T_j^{\text{fix}} = 0.02 \text{ s}$
Cloud aggregation profile	$\beta_C = 0.015 \text{ s/MB}$ and $T_C^{\text{fix}} = 0.04 \text{ s}$

model parameter partition, raw client decisions, link events, and round metrics in its output directory.

The system time used for efficiency comparisons is the modeled cloud-edge-end execution time plus the measured mode selection time. It includes the decision cost while ex-

cluding the sequential host overhead caused by emulating many clients on one machine. Host wall time is recorded separately as an implementation measurement and is not used as deployment latency.

5.2. CKKS Profile Validation

The CKKS profile uses polynomial modulus degree 8192, coefficient modulus bits [50, 40], scale 2^{40} , and chunks of at most 4096 values. A complete historical one round ResNet 18 validation selected HE for 40 clients and encrypted 21.13 GB of generated ciphertext. Encryption, ciphertext addition, and decryption took 335.78 s. The maximum absolute difference from plaintext aggregation was 9.17×10^{-8} . The profiled and encrypted executions produced the same test accuracy, and their

Table 6: ResNet 18 Execution and Aggregation Mapping

Mode	End execution	Edge execution	Cloud role
LIE	Frozen stem through layer2	Frozen downstream features; classifier training	Fixed DP/HE aggregate
LIC	Basic end/cloud split template	Relay	Excluded under current trust domain
LIIE	Frozen features; classifier training	Noisy packet aggregation	Fixed DP/HE aggregate
LIIC	Frozen features; classifier training	Noisy packet aggregation	Fixed DP/HE aggregate
LIEIIC	Frozen stem through layer2	Frozen features; classifier training	Fixed DP/HE aggregate
LIEIIC	Frozen stem through layer2	Three private classifier stages; one packet	Fixed DP/HE aggregate
LIEIIC	Three private classifier stages	One noisy packet	Fixed DP/HE aggregate

global update norms differed by 1.47×10^{-9} . These are historical full-model transport measurements, not validation results for the current classifier-only Method 2 integration. The 5130-coordinate payload requires a fresh CKKS cost calibration; old costs must not be presented as measured classifier-only system performance.

5.3. Compared Methods and Main Results

Ours performs joint Pareto search over the global client profile and selects a compromise between system latency and estimated convergence error cost. Individual Optimal uses the same feasible mode candidate set but selects the ideal point independently for each device without system coordination. Random samples from the same admissible mode set. These protected methods share the fixed global release contract. No Protection is reported only as a separately configured ablation/control run with the release gate disabled; it is not an admissible policy inside a formal mainline-fusion run.

Three fixed topology references place the framework beside established FL training structures. Fixed FedAvg uses local training followed by direct cloud aggregation through LIIC [1]. Fixed HFL uses local training, edge aggregation, and cloud aggregation through LIEIIC [2]. Fixed SplitFed uses end and edge split training followed by cloud aggregation through LIEIIC [11]. These references use the same fixed global release contract and all remaining settings as Ours. They are common topology implementations in the shared execution engine rather than reproductions of every component in the cited systems.

NSGA II uses the same feasible collaboration profiles and objective evaluator as Ours while replacing the bounded search with nondominated sorting, crowding selection, crossover, and mutation [30]. It provides a standard multiobjective optimizer reference under the same training and privacy conditions.

Table 7 and Fig. 5 present the three random seeds. The primary efficiency value is system time, which combines modeled workflow execution with measured decision time. The privacy column reports the accounted epsilon of the formal global aggregate release. The figure uses the common measured time system time interval determined by the shortest completed run and shows raw accuracy without temporal smoothing. Numerical values from execution revisions before `paper_flow_v30_mainline_fusion_method2_epochs` are not retained because they used different privacy, encrypted aggregation, feasible set, and timing semantics.

Table 7: Main comparison on CIFAR 10 with ResNet 18 over 100 communication rounds. Values will report the mean and 95% confidence interval over three random seeds

Method	Final Acc. (%)	Last 10 Acc. (%)	System Time (s)	Comm. (GB)	ϵ_{upd}
Ours	—	—	—	—	—
Individual Optimal	—	—	—	—	—
No Protection	—	—	—	—	—
Random	—	—	—	—	—
Fixed FedAvg	—	—	—	—	—
Fixed SplitFed	—	—	—	—	—
Fixed HFL	—	—	—	—	—
NSGA II	—	—	—	—	—

5.4. Reconfiguration Behavior

The client decision logs expose how the selected workflow changes during training. Fig. 6 reports the share of clients in each executable collaboration mode together with fixed global release protection coverage. The mode switch curve records the fraction of clients whose selected mode differs from the preceding round. These measurements distinguish dynamic workflow reconfiguration from a fixed topology with changing scalar parameters.

5.5. Second Dataset and Model

Fashion MNIST with LeNet 5 evaluates the comparison in a different image domain and model scale. The experiment uses the same 100 clients, 10 edge nodes, 100 communication rounds, three random seeds (42, 43, and 44), privacy targets, and collaboration policies. The local learning rate is 0.15 and each round uses one local epoch. The split is placed between the convolutional encoder and the fully connected classifier.

5.6. Decision Time and Scale

Fig. 8 reports measured decision time and system time with 20, 50, and 100 clients. Ours and Individual Optimal use the same client profiles, candidate sets, privacy targets, and training settings at each scale.

5.7. Small Scale Search Quality

We compare the bounded search with exact enumeration on 12 instances formed by 2 to 5 clients and random seeds 40, 42, and 44. Each client retains four representative feasible candidates. Both solvers use the same event flow evaluation, privacy conditions, and edge coverage constraint. Objective points with equal latency and error cost are counted once because different client assignments can be objective equivalent.

The bounded search recovers most unique Pareto objective points and reduces the number of evaluated profiles. The nonzero worst case gaps confirm that it is an approximate solver rather than an exact optimizer. The deterministic tie rule orders equal Tchebycheff values by latency, error cost, and profile identity.

5.8. Empirical Calibration of the Error Cost Estimate

For every Ours run, the value of $\widehat{\Omega}_{\text{sys}}^t$ is recorded before the selected round update. It is paired with the test loss after that update, the change in test loss, and the change in test accuracy. Spearman and Kendall correlations are computed within each seed and after standardizing each trajectory within its seed.

v30 main experiment figure pending

Figure 5: Raw test accuracy against system time on CIFAR 10 with ResNet 18. All curves use the common interval determined by the shortest completed run. The curves show the mean over three random seeds, the shaded regions show 95% confidence intervals, and the inset enlarges the final quarter of the common interval.

v30 reconfiguration figure pending

Figure 6: Raw collaboration mode shares, fixed global release protection coverage, and client mode switch rate over communication rounds. The curves report the mean over three random seeds.

Table 8: Fashion MNIST Results with LeNet 5 over 100 Communication Rounds

Method	Final Acc.	System Time	Comm.
Ours	—	—	—
Individual Optimal	—	—	—
No Protection	—	—	—
Random	—	—	—
Fixed FedAvg	—	—	—
Fixed SplitFed	—	—	—
Fixed HFL	—	—	—
NSGA II	—	—	—

This experiment evaluates whether the estimate orders observed training outcomes in the intended direction. A useful estimate is expected to correlate positively with post update loss and loss increase and negatively with accuracy increase.

v30 second dataset figure pending

Figure 7: Raw Fashion MNIST test accuracy against system time with LeNet 5. The curves report the mean and 95% confidence interval over three random seeds.

v30 scale experiment figure pending

Figure 8: Measured decision time and system time under different client scales.

Table 9: Bounded Search Quality Against Exact Enumeration

Metric	Mean	Worst
Pareto objective recall	96.73%	75.00%
Normalized hypervolume gap	0.0103	0.1158
Tchebycheff regret	0.0202	0.1137
Objective equivalent decision rate	83.33%	N/A
Evaluated profile reduction	34.90%	6.25%

5.9. Strategy Update Period

Fig. 9 evaluates strategy update periods 1, 5, 10, and 20 with seeds 40, 42, and 44. The experiment reports final and last 10 round accuracy, total decision time, and system time. This comparison tests how frequently the collaboration workflow must be reconfigured to retain its utility while controlling search overhead.

5.10. Privacy Budget Sensitivity

Fig. 10 evaluates total update privacy targets 1, 2, 4, and 8 with seeds 40, 42, and 44. Each run recalibrates the update noise multiplier for the same 100 round horizon. The figure reports realized client privacy loss, final accuracy, and last 10 round accuracy.

5.11. Ablation Study

The ablation study compares the complete method with three variants. No Global Coordination replaces the global profile search with independent client selection. No Error Cost Estimate removes $\hat{\Omega}_{\text{sys}}$ from the objectives. Fixed LIIEIIC

Table 10: Empirical Calibration of the Estimated Convergence Error Cost

Observed outcome	Spearman	Kendall
Post update test loss	—	—
Test loss change	—	—
Test accuracy change	—	—

v30 strategy period figure pending

Figure 9: Effect of the strategy update period on accuracy, decision time, and system time.

v30 privacy budget figure pending

Figure 10: Accuracy and realized privacy loss under different total privacy targets.

restricts every feasible client to the LIIEIIC mode while retaining the same training and feasibility checks.

The controlled comparison uses seeds 42, 43, and 44. The table reports the complete 100 round utility, communication, privacy, participation, and system time results. The figure shows raw accuracy within the common system time interval determined by the shortest completed ablation run.

6. Conclusion

This paper presented a dynamic cloud-edge-end collaboration reconfiguration framework for federated learning under resource and privacy constraints. The unified collaboration space represents execution placement, aggregation topology, transmitted objects, and return paths as the dynamic decision, while a fixed client-level DP plus HE contract protects every formal global release. The joint model evaluates latency, communication, fixed-protocol privacy noise, encrypted aggregation overhead, and estimated convergence error cost. The constrained multiobjective formulation is solved by bounded Pareto search and ideal point Tchebycheff selection.

The current framework uses profiled computation and link rates in its logical latency model, while the training process and

Table 11: Ablation results on CIFAR 10 with ResNet 18 over 100 communication rounds. Values will report the mean and 95% confidence interval over three random seeds

Variant	Final Acc. (%)	Last 10 Acc. (%)	System Time (s)	Comm. (GB)	Effective Clients (%)	ϵ_{upd}
Ours	—	—	—	—	—	—
No Global Coordination	—	—	—	—	—	—
No Error Cost Estimate	—	—	—	—	—	—
Fixed LIIIIIC	—	—	—	—	—	—

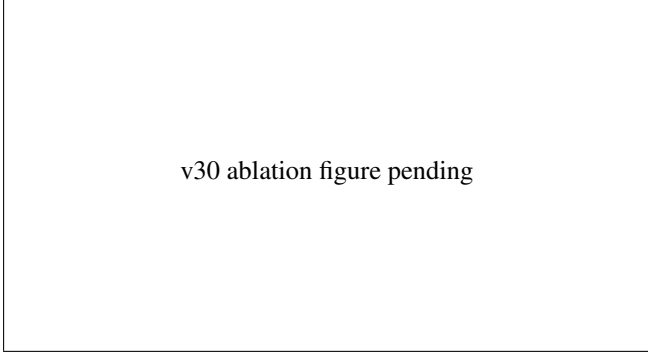


Figure 11: Raw test accuracy against system time for the complete method and three ablation variants. The curves use the common interval determined by the shortest completed run, and the inset enlarges the final quarter of that interval.

CKKS aggregation are executed on one host. Future work will combine online profiling with physical cloud-edge-end deployment and will further reduce reconfiguration overhead through parallel profile evaluation, distributed search, and warm starts.

Appendix A. Derivation of the Convergence Error Cost Estimate

This appendix provides the intermediate steps used to estimate convergence error cost in Section 3.2. It separates the effects of update clipping and Gaussian update noise.

Appendix A.1. PL Error Recursion

Let the deterministic parameter gradient perturbation caused by update clipping be

$$e_i^t = e_{\text{uclip},i}^t. \quad (\text{A.1})$$

For consistency with the separable upper bound used during selection, we use the conservative bound

$$\mathbb{E}[\|e_i^t\|_2^2 \mid w^t] \leq 3\mathbb{I}_{\text{upd}}^{i,t} \mathcal{B}_{\text{clip},i}^t. \quad (\text{A.2})$$

To make the coefficients in (24) explicit, define the effective round step size

$$\alpha = \eta_{\text{lr}} L_B \quad (\text{A.3})$$

and write one perturbed update in gradient units as

$$w^{t+1} = w^t - \alpha [\nabla F(w^t) + e_i^t + s_i^t], \quad (\text{A.4})$$

where e_i^t is the deterministic bias in (A.1) and s_i^t is conditionally zero mean stochastic error. Let

$$\begin{aligned} V_i^t &= \mathbb{E}[\|s_i^t\|_2^2 \mid w^t] \\ &\leq \sigma_{\text{local}}^2 + m_{u,i}^2 \frac{\sigma_u^2 \Delta_u^2 d}{\alpha^2}. \end{aligned} \quad (\text{A.5})$$

Dividing the update perturbation by α converts it to gradient units, which explains the factor α^{-2} in (A.5). After admission, the client terms are combined using $q_i^t(\mathbf{x})$. The independent variance terms are therefore multiplied by $(q_i^t)^2$, as shown in (23). Noise added after edge aggregation is combined using the corresponding group weight q_g^t .

Let $g^t = \nabla F(w^t)$, and condition on w^t . By G -smoothness and $\mathbb{E}[s_i^t \mid w^t] = 0$,

$$\begin{aligned} \mathbb{E}_t[F(w^{t+1})] &\leq F(w^t) - \alpha \langle g^t, g^t + e_i^t \rangle \\ &\quad + \frac{G\alpha^2}{2} (\|g^t + e_i^t\|_2^2 + V_i^t). \end{aligned} \quad (\text{A.6})$$

Using the exact identity

$$\langle g, g + e \rangle = \frac{1}{2} \|g\|_2^2 + \frac{1}{2} \|g + e\|_2^2 - \frac{1}{2} \|e\|_2^2, \quad (\text{A.7})$$

and assuming $0 < \alpha \leq 1/G$, (A.6) becomes

$$\begin{aligned} \mathbb{E}_t[F(w^{t+1})] &\leq F(w^t) - \frac{\alpha}{2} \|g^t\|_2^2 - \frac{\alpha}{2} (1 - G\alpha) \|g^t + e_i^t\|_2^2 \\ &\quad + \frac{\alpha}{2} \|e_i^t\|_2^2 + \frac{G\alpha^2}{2} V_i^t \\ &\leq F(w^t) - \frac{\alpha}{2} \|g^t\|_2^2 + \frac{\alpha}{2} \|e_i^t\|_2^2 + \frac{G\alpha^2}{2} V_i^t. \end{aligned} \quad (\text{A.8})$$

The PL inequality implies $-\alpha \|g^t\|_2^2/2 \leq -\mu\alpha[F(w^t) - F(w^*)]$. Define

$$\rho_{\text{PL}} = \mu\alpha = \mu\eta_{\text{lr}} L_B. \quad (\text{A.9})$$

Substituting (A.2) into (A.8) therefore gives

$$\begin{aligned} \mathbb{E}_t[F(w^{t+1}) - F(w^*)] &\leq (1 - \rho_{\text{PL}})[F(w^t) - F(w^*)] \\ &\quad + \frac{3\alpha}{2} S_{\text{bias},i}^t + \frac{G\alpha^2}{2} V_i^t, \end{aligned} \quad (\text{A.10})$$

where

$$S_{\text{bias},i}^t = \mathbb{I}_{\text{upd}}^{i,t} \mathcal{B}_{\text{clip},i}^t. \quad (\text{A.11})$$

Because $G\alpha^2 V_i^t/2 \leq G\alpha^2 V_i^t$, we use the conservative residual

$$\mathcal{R}(h_i) = \frac{3\alpha}{2} S_{\text{bias},i}^t + G\alpha^2 V_i^t. \quad (\text{A.12})$$

For a fixed conditional residual, iterating the recursion gives

$$\begin{aligned} \mathbb{E}[F(w^t) - F(w^*)] &\leq (1 - \rho_{\text{PL}})^t [F(w^0) - F(w^*)] \\ &\quad + \frac{1 - (1 - \rho_{\text{PL}})^t}{\rho_{\text{PL}}} \mathcal{R}(h_i). \end{aligned} \quad (\text{A.13})$$

Thus its persistent contribution is estimated by $\mathcal{R}(h_i)/\rho_{\text{PL}}$. Substituting $\alpha = \eta_{\text{lr}} L_B$, (A.5), and (A.11) gives the client terms B_i^t and V_i^t in (24). Update terms are multiplied by their numbers of formal releases. In the fused mainline the Gaussian release is applied once to the weighted aggregate with sensitivity from (16); there is no mode-selectable client-versus-edge DP mechanism. Weighting client clipping-bias terms by q_i^t and adding the common aggregate-noise variance gives (26).

Appendix B. CKKS Implementation and Security Scope

The HE path uses the CKKS approximate arithmetic scheme [31]. In the intended protocol, clients and edge nodes use the public key to encrypt an HE protected update. The cloud evaluator performs ciphertext addition, and a separate trusted coordinator holds the secret key and decrypts the aggregate. In the formal fused SEAL path, the Method 2 custodian authorizes a complete edge cohort, starts a ciphertext-only cloud worker, verifies its full ciphertext sum, and decrypts the already-noisy result once. A fresh key and custodian session are created for each global release; the persistent global release accountant tracks the FL horizon across checkpoints. The secret key is not sent to the cloud worker. The custodian is trusted and can bypass its own policy; the local process boundary is not an OS sandbox, threshold decryption, or an independently deployed cloud-edge system.

The implementation uses polynomial modulus degree 8192, coefficient modulus bits [50, 40], scale 2^{40} , and chunks of at most 4096 values. Every release logs ciphertext bytes, encrypted coordinate count, operation time, and maximum absolute error against the weighted noisy plaintext aggregate. Profiled plaintext execution is a legacy control and is rejected by the formal fused configuration.

Code and Data Availability

The supplementary artifact contains the implementation, versioned experiment configurations, generated device profiles, exact data partitions, and raw round logs used in this study. The datasets are obtained through their public torchvision interfaces and are not redistributed.

References

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial intelligence and statistics, PMLR, 2017, pp. 1273–1282.
- [2] L. Liu, J. Zhang, S. Song, K. B. Letaief, Client-edge-cloud hierarchical federated learning, in: ICC 2020 - 2020 IEEE International Conference on Communications (ICC), 2020, pp. 1–6. doi:10.1109/ICC40277.2020.9148862.
- [3] T.-M. H. Hsu, H. Qi, M. Brown, Measuring the effects of non-identical data distribution for federated visual classification, arXiv preprint arXiv:1909.06335 (2019).
- [4] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, Proceedings of Machine learning and systems 2 (2020) 429–450.
- [5] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, K. Chan, Adaptive federated learning in resource constrained edge computing systems, IEEE Journal on Selected Areas in Communications 37 (6) (2019) 1205–1221. doi:10.1109/JSAC.2019.2904348.
- [6] C. Ma, X. Li, B. Huang, G. Li, F. Li, Personalized client-edge-cloud hierarchical federated learning in mobile edge computing, Journal of Cloud Computing 13 (12) (2024). doi:10.1186/s13677-024-00721-w.
- [7] T. Du, X. Chen, G. Kou, Y. Zhao, C. Xu, A clustering federated learning method for cloud-edge personalized model decoupling, Chinese Journal of Computers 48 (2) (2025) 407–432.
- [8] C. T. Dinh, N. H. Tran, M. N. H. Nguyen, C. S. Hong, W. Bao, A. Y. Zomaya, V. Gramoli, Federated learning over wireless networks: Convergence analysis and resource allocation, IEEE/ACM Transactions on Networking 29 (1) (2021) 398–409. doi:10.1109/TNET.2020.3035770.
- [9] Z. Wu, S. Sun, Y. Wang, M. Liu, B. Gao, Q. Pan, T. He, X. Jiang, Agglomerative federated learning: Empowering larger model training via end-edge-cloud collaboration, IEEE INFOCOM 2024 - IEEE Conference on Computer Communications (2023) 131–140.
URL <https://api.semanticscholar.org/CorpusID:266351000>
- [10] F. Mazloomi, S. Shah-Heydari, K. El-Khatib, A novel multi-server federated learning framework in vehicular edge computing, Future Internet 17 (2025) 315.
URL <https://api.semanticscholar.org/CorpusID:280304434>
- [11] J. Haga, Y. Tanimura, T. T. Nguyen, et al., Sfetec: Split-federated learning scheme optimized for thing-edge-cloud environment, in: 2024 IEEE 20th International Conference on e-Science (e-Science), IEEE, 2024, pp. 1–2.
- [12] J. Chen, H. Jiang, Q. Hu, Utility-enhanced personalized privacy preservation in hierarchical federated learning, IEEE Transactions on Mobile Computing 24 (6) (2025) 5264–5279. doi:10.1109/TMC.2025.3531919.
- [13] X. Xu, Y. Li, L. Lu, T. Wang, Adpfl: Adaptive differential privacy-enhanced federated learning, IEEE Internet of Things Journal 12 (23) (2025) 50682–50692. doi:10.1109/JIOT.2025.3611410.
- [14] E. Elabd, Dynamic differential privacy technique for deep learning models, Scientific Reports 15 (2025) 39353. doi:10.1038/s41598-025-27708-0.
- [15] S. Dong, R. Hou, Privacy-preserving task offloading scheme based on blockchain federated learning in end-edge-cloud computing environments, IEEE Internet of Things Journal 13 (13) (2026) 28810–28824. doi:10.1109/JIOT.2026.3684948.
- [16] A. S. Kalafatelis, V. Nikolakakis, N. Tsoulakos, P. Trakadas, Privacy-preserving hierarchical federated learning over data spaces, in: 2025 IEEE International Conference on Big Data (BigData), 2025, pp. 6426–6435. doi:10.1109/BigData66926.2025.11400909.

- [17] M. M. Islam, W. M. Abdullah, B. N. Saha, Privacy-preserving hierarchical fog federated learning (pp-hffl) for iot intrusion detection, *Sensors* (Basel, Switzerland) 25 (2025).
URL <https://api.semanticscholar.org/CorpusID:283618612>
- [18] M. F. Pervej, R. Jin, H. Dai, Hierarchical federated learning in wireless networks: Pruning tackles bandwidth scarcity and system heterogeneity, *IEEE Transactions on Wireless Communications* 23 (9) (2024) 11417–11432.
- [19] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, J. Liu, Adaptive control of local updating and model compression for efficient federated learning, *IEEE Transactions on Mobile Computing* 22 (10) (2023) 5675–5689. doi:10.1109/TMC.2022.3186936.
- [20] J. Chen, H. Hou, Y. Hou, Z. Li, Hermes: Multi-job federated learning with switching cost in wireless networks, *Computer Networks* 281 (2026) 112204. doi:<https://doi.org/10.1016/j.comnet.2026.112204>.
URL <https://www.sciencedirect.com/science/article/pii/S1389128626002161>
- [21] J. Wang, N. Wang, R. Zhou, B. Li, Efficient and intelligent multijob federated learning in wireless networks, *IEEE Internet of Things Journal* 12 (7) (2025) 8685–8698. doi:10.1109/JIOT.2024.3502403.
- [22] C. Zhang, Z. Zheng, Task migration for mobile edge computing using deep reinforcement learning, *Future Generation Computer Systems* 96 (2019) 111–118. doi:<https://doi.org/10.1016/j.future.2019.01.059>.
URL <https://www.sciencedirect.com/science/article/pii/S0167739X18329674>
- [23] X. Zhu, W. Yao, W. Wang, Load-aware task migration algorithm toward adaptive load balancing in edge computing, *Future Generation Computer Systems* 157 (2024) 303–312. doi:<https://doi.org/10.1016/j.future.2024.03.014>.
URL <https://www.sciencedirect.com/science/article/pii/S0167739X24000864>
- [24] Z. Zhou, S. Shi, F. Wang, Y. Zhang, Y. Li, Joint client selection and cpu frequency control in wireless federated learning networks with power constraints, *Entropy* 25 (8) (2023) 1183. doi:10.3390/e25081183.
URL <https://www.mdpi.com/1099-4300/25/8/1183>
- [25] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He, Zero: memory optimizations toward training trillion parameter models, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '20*, IEEE Press, 2020.
- [26] L. Li, J. Lv, S. Wang, C. Hu, G. Sun, N. Li, Latency and energy-aware adaptive service migration in mobile edge computing, *Scientific Reports* 16 (1) (2026) 6178.
- [27] I. Mironov, Rényi differential privacy, in: *2017 IEEE 30th Computer Security Foundations Symposium*, 2017, pp. 263–275. doi:10.1109/CSF.2017.11.
- [28] X. Zhang, X. Chen, M. Hong, S. Wu, J. Yi, Understanding clipping for Federated Learning: Convergence and Client-Level Differential Privacy, in: K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, S. Sabato (Eds.), *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162 of *Proceedings of Machine Learning Research*, PMLR, 2022, pp. 26048–26067.
URL <https://proceedings.mlr.press/v162/zhang22b.html>
- [29] R. E. Steuer, E.-U. Choo, An interactive weighted Tchebycheff procedure for multiple objective programming, *Mathematical Programming* 26 (3) (1983) 326–344. doi:10.1007/BF02591870.
URL <https://doi.org/10.1007/BF02591870>
- [30] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: Nsga-ii, *IEEE Transactions on Evolutionary Computation* 6 (2) (2002) 182–197. doi:10.1109/4235.996017.
- [31] J. H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, in: *Advances in Cryptology, ASIACRYPT 2017*, Springer, 2017, pp. 409–437. doi:10.1007/978-3-319-70694-8_15.